

# 中国科学院大学

## 《计算机体系结构(研讨课)》实验报告

姓名 袁晨圃 学号 2023K8009929012 专业 计算机科学与技术  
实验项目编号 7,8,9 实验名称 流水线 CPU 设计

### 一、 逻辑电路结构与仿真波形的截图及说明

#### 1 实验 8

##### 1.1 处理器的结构设计框图

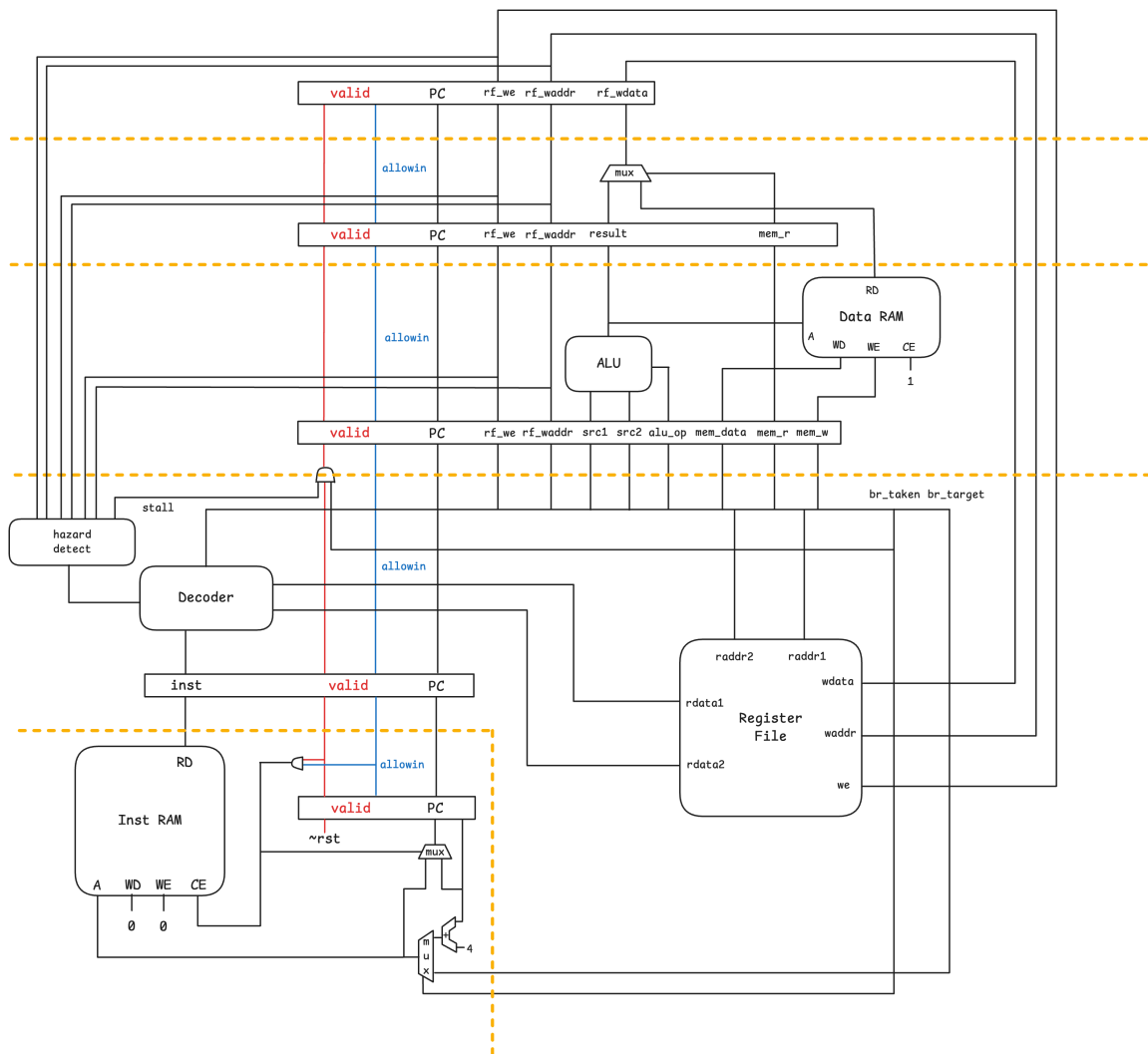


图 1: exp8 处理器结构

##### 1.2 关键部件设计

- 流水线控制模块

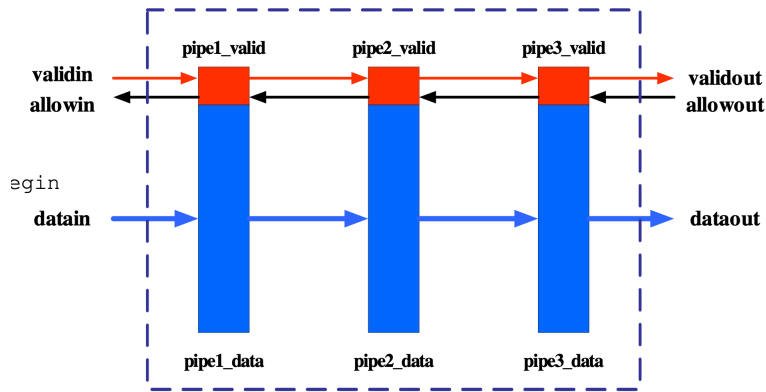


图 2: 流水线示意图

如图所示, 每一级流水线都需要实现一组流水线控制信号, 这对于不同流水级而言是重复代码, 因此我选择例化一个流水线模块, 负责生成流水线信号。

这样做方便对于流水线功能的统一维护。

```
module pipeline(
    input wire clk, rst,
    input wire allowout, // -->? [next_stage]
    input wire validin, // valid? ---> [stage]
    input wire readygo, // [stage] -->?
    output wire validout, // [stage] --> valid?
    output wire allowin, // --->? [stage]
    output reg valid // [stage?]
);

assign allowin = ~valid | (readygo & allowout);
assign validout = valid & readygo;

always @(posedge clk) begin
    if (rst) begin
        valid <= 1'b0;
    end else if (allowin) begin
        valid <= validin;
    end
end

wire refreshing = validin & allowin;
endmodule
```

validin, allowin 是与前一级交互的数据

validout, allowout 是与后一级交互的数据

当前流水级的 valid 信号表示当前流水级是否有有效数据

readygo 信号表示当前流水级是否准备好将数据传递到下一级, 由外部传入, 然后流水线模块根据 readygo 信号生成 allowin 和 validout。

refreshing 信号表示当前流水级和前一级握手成功, 刷新当前流水级数据

#### ● 冲突检测模块

```

wire conflict1 =
    rf_raddr1 == 5'h0 ? 0 :
    (u_stage_ex.valid && u_stage_ex.output_rf_we && (u_stage_ex.output_rf_waddr == rf_raddr1)) ||
    (u_stage_mem.valid && u_stage_mem.output_rf_we && (u_stage_mem.output_rf_waddr == rf_raddr1))
    ||
    (u_stage_wb.valid && u_stage_wb.output_rf_we && (u_stage_wb.output_rf_waddr == rf_raddr1));
wire conflict2 =
    rf_raddr2 == 5'h0 ? 0 :
    (u_stage_ex.valid && u_stage_ex.output_rf_we && (u_stage_ex.output_rf_waddr == rf_raddr2)) ||
    (u_stage_mem.valid && u_stage_mem.output_rf_we && (u_stage_mem.output_rf_waddr == rf_raddr2))
    ||
    (u_stage_wb.valid && u_stage_wb.output_rf_we && (u_stage_wb.output_rf_waddr == rf_raddr2));

assign id_stall = conflict1 || conflict2;

```

根据后三个流水级的写地址以及写地址跟 ID 流水级读地址的比较,判断是否存在数据冒险,然后阻塞 IF 和 ID 流水级

## 2 实验 9

### 2.1 处理器的结构设计框图

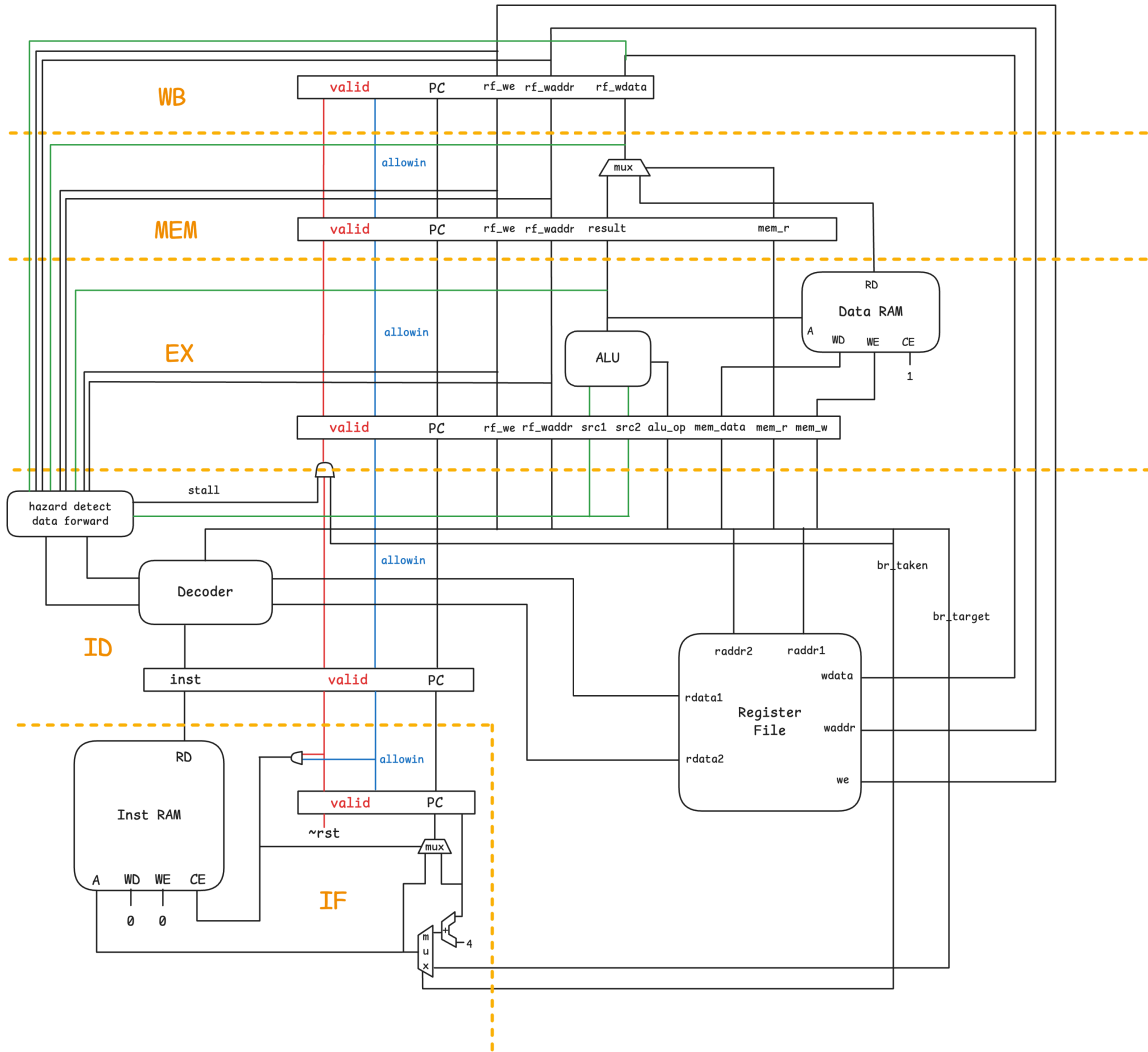


图 3: 加入了前递模块的处理器结构

### 2.2 关键部件设计

#### • 数据前递模块

功能: 分析后三个流水级的写地址以及写使能信号, 判断是否需要前递数据, 在 ID 流水级生成前递后的寄存器读数据。

```
wire hazard_ex1 = ex_wen && (u_stage_ex.output_rf_waddr == rf_raddr1);
wire hazard_ex2 = ex_wen && (u_stage_ex.output_rf_waddr == rf_raddr2);
wire hazard_mem1 = mem_wen && (u_stage_mem.output_rf_waddr == rf_raddr1);
wire hazard_mem2 = mem_wen && (u_stage_mem.output_rf_waddr == rf_raddr2);
wire hazard_wb1 = wb_wen && (u_stage_wb.output_rf_waddr == rf_raddr1);
wire hazard_wb2 = wb_wen && (u_stage_wb.output_rf_waddr == rf_raddr2);
wire [31:0] ex_forward_data;
wire [31:0] mem_forward_data;
wire [31:0] wb_forward_data;
```

```

wire ex_forward_ready;
wire mem_forward_ready;
wire wb_forward_ready;

assign rf_rdata1 = hazard_ex1 ? ex_forward_data :
                    hazard_mem1 ? mem_forward_data :
                    hazard_wb1 ? wb_forward_data :
                    raw_rf_rdata1;

assign rf_rdata2 = hazard_ex2 ? ex_forward_data :
                    hazard_mem2 ? mem_forward_data :
                    hazard_wb2 ? wb_forward_data :
                    raw_rf_rdata2;

wire id_stall1 = hazard_ex1 ? ~ex_forward_ready :
                 hazard_mem1 ? ~mem_forward_ready :
                 hazard_wb1 ? ~wb_forward_ready :
                 1'b0;

wire id_stall2 = hazard_ex2 ? ~ex_forward_ready :
                 hazard_mem2 ? ~mem_forward_ready :
                 hazard_wb2 ? ~wb_forward_ready :
                 1'b0;

wire id_stall = id_stall1 | id_stall2;

```

这些 \*\_wen 信号是比较 tricky 的地方。

由于对 0 寄存器的写数据不应该前递,我直接让 ex\_wen,mem\_wen,wb\_wen 在写地址为 0 时为 0。

直观的理解:写地址为 0 时其实相当于没有写寄存器,因此不需要前递。

```

wire ex_wen = u_stage_ex.valid && u_stage_ex.output_rf_we && u_stage_ex.output_rf_waddr != 5'h0;
wire mem_wen = u_stage_mem.valid && u_stage_mem.output_rf_we && u_stage_mem.output_rf_waddr !=
               5'h0;
wire wb_wen = u_stage_wb.valid && u_stage_wb.output_rf_we && u_stage_wb.output_rf_waddr != 5'h0;

```

为了一般性考虑,我为每一个流水级都设置了一个 forward\_ready 信号,表示当前流水级的前递数据是否准备好。当前设计中,forward\_ready 信号只会在 load 指令处在 EX 级时为 0 (因为需要等待到 MEM 级获取内存数据),其他情况均为 1。

随着设计的复杂,forward\_ready 信号可能会包含更复杂的逻辑,这里提前预留好重构的接口,增加逻辑只需要改对应流水级的 forward\_ready 信号生成逻辑即可。

EX, MEM, WB 前递的 data 分别来自于 EX 级的 ALU 结果,MEM 级的 ALU 结果或内存读数据,WB 级的写回数据。

```

// in stage-ex:
assign forward_data = alu_result;
assign forward_ready = ~output_mem_read; // NOTE: not ready only if needs to read memory
// in stage-mem:
assign output_rf_wdata = mem_read ? mem_rdata : alu_result;
assign forward_data = output_rf_wdata;

```

```

    assign forward_ready = 1'b1;
// in stage-wb:
    assign forward_data = rf_wdata;
    assign forward_ready = 1'b1;

```

## 2.3 前递功能效果展示

```

----[1366305 ns] Number 8'd20 Functional Test Point PASS!!!
=====
Test end!
----PASS!!!
$finish called at time : 1367295 ns : File "/Users/ycp/Source/Courses/ca-lab/cdp_edc_local/exp8/soc_verify/soc_bram/testbench/mycpu_tb.v" Line 270
run: Time (s): cpu = 00:00:16 ; elapsed = 00:00:13 . Memory (MB): peak = 8509.219 ; gain = 41.785 ; free physical = 5053 ; free virtual = 15185

```

图 4: exp8 运行时间:1367295 ns

```

----[ 604135 ns] Number 8'd20 Functional Test Point PASS!!!
=====
Test end!
----PASS!!!
$finish called at time : 604585 ns : File "/Users/ycp/Source/Courses/ca-lab/cdp_edc_local/exp9/soc_verify/soc_bram/testbench/mycpu_tb.v" Line 270
run: Time (s): cpu = 00:00:14 ; elapsed = 00:00:11 . Memory (MB): peak = 8634.121 ; gain = 48.676 ; free physical = 5016 ; free virtual = 15139

```

图 5: exp9 运行时间:604585 ns

## 二、 实验过程中遇到的问题、对问题的思考过程及解决方法

### • 取指阶段对于同步 RAM 时序的理解问题

问题复现:

```

(bug on 2097 ns)
/* source asm code */
1c000000 <_start>:
kernel_entry():
1c000000: 02bffc0c   addi.w $r12,$r0,-1(0xffff)
1c000004: 02bffc0c   addi.w $r12,$r0,-1(0xffff)
1c000008: 50fff800   b 65528(0xffff8) # 1c010000 <locate>

/* buggy instruments fetched by IF stage */
00000: 02bffc0c
00004: 02bffc0c
00008: 02bffc0c

```

分析了一下当前的代码逻辑,总结时序如下:

```

cycle0:
    input_pc => pc
    inst_sram_raddr = input_pc
    inst_sram_rdata => inst

cycle1:
    output_pc = pc
    output_inst = inst

```

可以看到,其实还是相当于异步 ram 了,同步 ram 的话应该把 inst\_addr 输出放在 IF 级之前

```
cycle0:
    next_pc = br_taken ? br_target : (preIF.pc + 4)
    inst_addr = next_pc
    next_pc => preIF.pc
cycle1:
    preIF.pc => IF.pc
    inst_data => IF.inst // get inst data after one cycle
cycle2:
    IF.output_pc = IF.pc
    IF.output_inst = IF.inst
```

思考:这样的话 IF 流水级本身不保存指令地址,要求 ram 在没有新请求之前保持原来的值,否则流水线阻塞的时候会不会有问题?

解决方案:控制 Inst SRAM 的 ce 信号,只有当有指令进入 ID 级时才拉高 ce,同时向 Inst SRAM 传递的指令地址是 next\_pc,ID 级直接从 inst\_sram\_rdata 读取指令。

```
always @(posedge clk) begin
    if (rst) begin
        pc <= ENTRYPOINT;
    end else if (id_refreshing) begin
        pc <= nextpc;
    end
end
assign inst_sram_en = rst | id_refreshing;
```

这样可以保证 PC 和 inst\_sram\_rdata 的值始终是对应的。

id\_refreshing = if.validout & id.allowin,如图 Fig. 3 所示。

#### • br\_taken 设置问题

id\_stall 时不能拉高 br\_taken,因为此时读取的值还不对。

### 三、 实验所耗时间

在课后,花费了大约\_\_\_\_10\_\_\_\_ 小时完成此次实验。